

KnowledgeHub AI Project Documentation

Overview

KnowledgeHub AI is a two-part application consisting of a FastAPI backend and a Vite + React frontend. It supports user authentication, document upload, semantic search, and chat-based question answering grounded in uploaded documents.

Repository Structure

- backend/: FastAPI application code, database models, authentication, document handling, embeddings, and chat endpoint.
- frontend/: React user interface built with Vite and Axios for backend communication.
- start-app.sh: launcher script to start backend and frontend together.

Setup Requirements

System requirements:

- Python 3.9+
- Node.js 18+
- npm
- PostgreSQL
- tesseract and poppler for PDF OCR support.

Backend dependencies:

- fastapi
- uvicorn[standard]
- python-jose[cryptography]
- passlib[bcrypt]
- python-multipart
- pydantic
- pydantic-settings
- email-validator
- sqlalchemy
- psycopg2-binary
- requests
- numpy
- PyPDF2
- pytesseract
- pdf2image
- sentence-transformers
- chromadb

Frontend dependencies:

- react
- react-dom
- react-router-dom

- axios
- vite
- @vitejs/plugin-react

Backend Architecture

Entry point:

- backend/app/main.py initializes the FastAPI app, configures CORS, imports router modules, and initializes the database.

Authentication:

- backend/app/auth/routes.py: registration and login endpoints.
- backend/app/auth/schemas.py: user and token Pydantic models.
- backend/app/auth/utils.py: password hashing and verification.
- backend/app/auth/deps.py: JWT validation and current-user dependency.
- backend/app/auth/service.py: user creation and lookup logic.

Database:

- backend/app/db/database.py: SQLAlchemy engine, session, and model base.
- backend/app/db/init_db.py: creates tables on startup.
- backend/app/models.py: ORM models for users, documents, chat sessions, and messages.

Configuration:

- backend/app/core/config.py: environment-backed settings for database, auth, LLM, and pipeline tuning.

Document and chat processing:

- backend/app/docs/routes.py: document upload, list, delete, semantic search, session management, and chat endpoints.
- backend/app/docs/chat_utils.py: prompt generation and context formatting.
- backend/app/docs/chunking.py: text chunking utility used before embedding.
- backend/app/docs/response_utils.py: answer sanitization, trimming, and greeting detection.

Embeddings:

- backend/app/embeddings/hf_client.py: manages embedding backend selection and ChromaDB vector storage.

Frontend Architecture

Entry point:

- frontend/src/main.jsx: renders the React app inside BrowserRouter.

Main application:

- frontend/src/App.jsx: handles authentication, document upload, dashboard, chat UI, sessions, and backend communication.

Backend API integration:

- Uses VITE_API_URL or defaults to http://localhost:8000.
- Auth: /api/auth/login, /api/auth/register.
- Documents: /api/docs/, /api/docs/upload, /api/docs/{doc_id}.
- Search: /api/docs/search.

- Sessions: `/api/docs/sessions`, `/api/docs/sessions/{session_id}`.
- Chat stream: `/api/docs/chat/stream`.

Chat streaming:

- The frontend uses `fetch` with `response.body.getReader()` to consume SSE-like streamed tokens from the backend.

Running the Project

Launcher script:

- Run `./start-app.sh` from project root to start both backend and frontend.

Manual startup:

Backend:

- `cd backend`
- `source .venv/bin/activate`
- `export PYTHONPATH="$PWD"`
- `python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000`

Frontend:

- `cd frontend`
- `npm install`
- `npm run dev -- --host 127.0.0.1 --port 5173`

Environment Configuration

Use `backend/.env.example` to create `backend/.env`. Important variables included `DATABASE_URL`, `SECRET_KEY`, `LLM_PROVIDER`, `OLLAMA_BASE_URL`, `HF_API_KEY`, and `HF_INFERENCE_ENDPOINT`.

Operational Notes

- Uploaded files save to `backend/uploads` and are vector indexed in ChromaDB.
- PDF extraction uses PyPDF2, with OCR fallback via `pdf2image` and `pytesseract` when necessary.
- Chat sessions and messages are stored for contextual response retrieval.

Deployment Considerations

- Use a secure `SECRET_KEY` in production.
- Use a secure PostgreSQL database and credentials.
- Serve frontend and backend behind HTTPS in production.

Testing and Validation

- Register and login successfully.
- Upload PDF/TXT documents and verify indexing.
- Ask questions and confirm the chat stream returns grounded answers.

Fixes Applied During Current Work

- Added explicit CORS origins for localhost and 127.0.0.1 in `backend/app/main.py`.
- Fixed missing `sanitize_answer` import in `backend/app/docs/routes.py` that caused `/chat/stream` to return 500 errors.